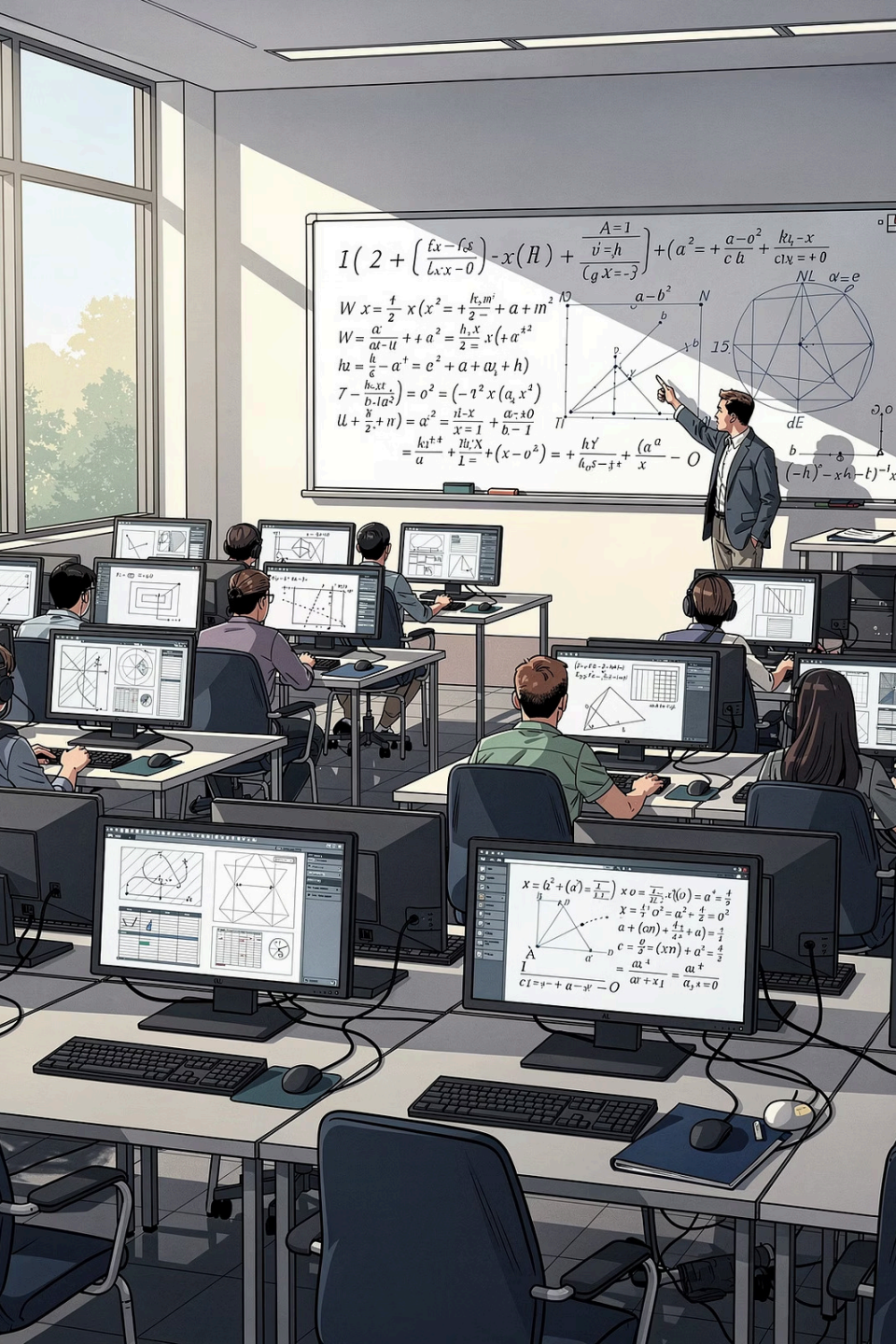




Numerical Computing with NumPy

Course 040223101 · Computer Programming for Mathematics 1 · Chapter 10

Department of Mathematics, Faculty of Applied Science
King Mongkut's University of Technology North Bangkok

Instructor: Assoc. Prof. Dr. Anuchit Jitpattanakul · Semester 1, Academic Year
2569

Chapter 10 Learning Objectives

Upon completing this chapter, students will be able to:

O1

Explain NumPy Array Advantages

Describe the advantages of NumPy arrays over Python lists for numerical computation tasks.

O2

Create 1D and 2D Arrays

Construct one-dimensional arrays and two-dimensional arrays (matrices) using NumPy functions.

O3

Perform Vectorized Computation

Apply vectorized calculations to arrays without writing explicit loops, leveraging NumPy's core strength.

O4

Matrix Operations

Perform matrix operations including addition, multiplication, and transpose using NumPy operators.

O5

Apply NumPy to Math Problems

Use NumPy to solve mathematical problems such as systems of linear equations with `np.linalg.solve`.

 This chapter's content aligns with Course Learning Outcome **CLO 5**.



Chapter 10: Numerical Computing with NumPy

Mathematical and scientific computation frequently involves large datasets, vectors, and matrices. **NumPy (Numerical Python)** is a library designed specifically for these tasks. It provides a data structure – the **ndarray** – that computes faster and is written more concisely than using loops with Python lists.

This chapter is the **final chapter** of the course, integrating all prior knowledge – variables, conditions, loops, data structures, and functions – into the domain of numerical computing. It forms an essential foundation for higher-level studies in scientific computing and data science.

10.1 Introduction to NumPy

What is NumPy?

NumPy is an external library that must be installed before use. It is typically installed with `pip install numpy` or comes pre-installed with Anaconda. It is conventionally imported with the alias `np`.

The core structure is the **ndarray** (n-dimensional array), which stores elements of the same data type and supports whole-array computation in a single operation.

First NumPy Program – lOarrayl.py

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(a)
print(type(a))
print("จำนวนสมาชิก:", a.size)
```

Output

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
จำนวนสมาชิก: 5
```


Why NumPy Arrays Over Python Lists?

Homogeneous Data Type

All elements in a NumPy array share the same data type, enabling efficient memory storage and faster computation compared to Python lists which can hold mixed types.

Vectorized Operations

Mathematical operations apply to every element simultaneously without writing explicit `for` loops. This results in shorter, more readable code.

Speed

NumPy operations are implemented in optimized C code under the hood, making them significantly faster than equivalent Python loop-based computations on large datasets.

Rich Mathematical Functions

NumPy provides a comprehensive library of mathematical, statistical, and linear algebra functions that operate directly on arrays with minimal code.

10.2 Creating NumPy Arrays

NumPy provides several built-in functions for creating arrays. Each serves a different purpose depending on the type of data needed.

Command	Meaning	Example Output
<code>np.array([1,2,3])</code>	Create from a Python list	<code>[1 2 3]</code>
<code>np.zeros(3)</code>	Array of three zeros	<code>[0. 0. 0.]</code>
<code>np.ones(3)</code>	Array of three ones	<code>[1. 1. 1.]</code>
<code>np.arange(0,10,2)</code>	Sequence like Python <code>range</code>	<code>[0 2 4 6 8]</code>
<code>np.linspace(0,1,5)</code>	5 equally spaced values in <code>[0,1]</code>	<code>[0. 0.25 0.5 0.75 1.]</code>

Array Creation – Code Example

File: lOcreate.py

```
import numpy as np

print(np.zeros(4))
print(np.arange(1, 6))
print(np.linspace(0, 10, 5))
```

Output

```
[0. 0. 0. 0.]
[1 2 3 4 5]
[ 0.  2.5  5.  7.5 10.]
```

Key Differences: arange vs linspace

`np.arange(start, stop, step)`

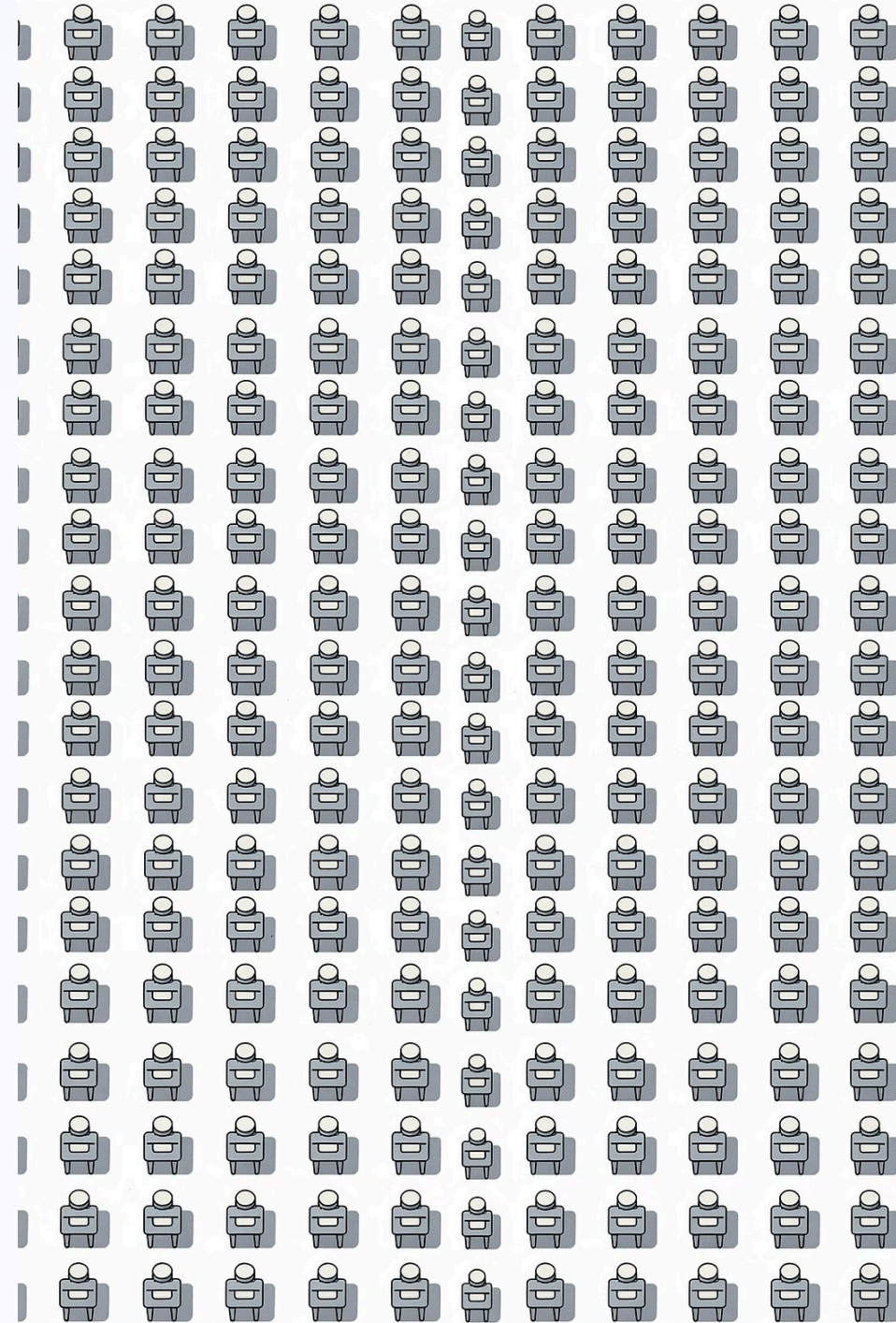
Generates values from `start` up to (but not including) `stop`, incrementing by `step`. Similar to Python's built-in `range()` but returns an ndarray. The number of elements depends on the step size.

`np.linspace(start, stop, num)`

Generates exactly `num` evenly spaced values between `start` and `stop` (inclusive). Preferred when you need a specific count of points, e.g., for plotting or numerical integration.

10.3 Vectorized Computation

The defining strength of NumPy is **vectorization** – performing a mathematical operation on every element of an array simultaneously, without writing a loop. Operators such as `+`, `*`, and `**` act element-wise across the entire array in a single expression.



Vectorized Operations – Code Example

File: lOvector.py

```
import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])

print(a + b)
print(a * 2)
print(a ** 2)
print(np.sqrt(a))
```

Output

```
[11 22 33 44]
[2 4 6 8]
[ 1  4  9 16]
[1.  1.41421356 1.73205081 2.  ]
```

What Each Operation Does

- `a + b` – adds corresponding elements: $1+10=11$, $2+20=22$, etc.
- `a * 2` – multiplies every element by the scalar 2 (broadcasting).
- `a ** 2` – raises each element to the power of 2 (squares).
- `np.sqrt(a)` – applies square root to each element using a universal function (ufunc).

Statistical Functions on Arrays

NumPy includes ready-made statistical functions that operate directly on arrays, eliminating the need to write manual aggregation loops.

File: lOstat.py

```
import numpy as np

data = np.array([4, 8, 15, 16, 23, 42])

print("ผลรวม:", data.sum())
print("เฉลี่ย:", data.mean())
print("ส่วนเบี่ยงเบนมาตรฐาน:",
      round(data.std(), 4))
print("สูงสุด/ต่ำสุด:",
      data.max(), data.min())
```

.sum()

Sum of all elements:
 $4+8+15+16+23+42 = \mathbf{108}$

.mean()

Arithmetic mean: $108 \div 6 = \mathbf{18.0}$

.std()

Population standard deviation:
12.3153

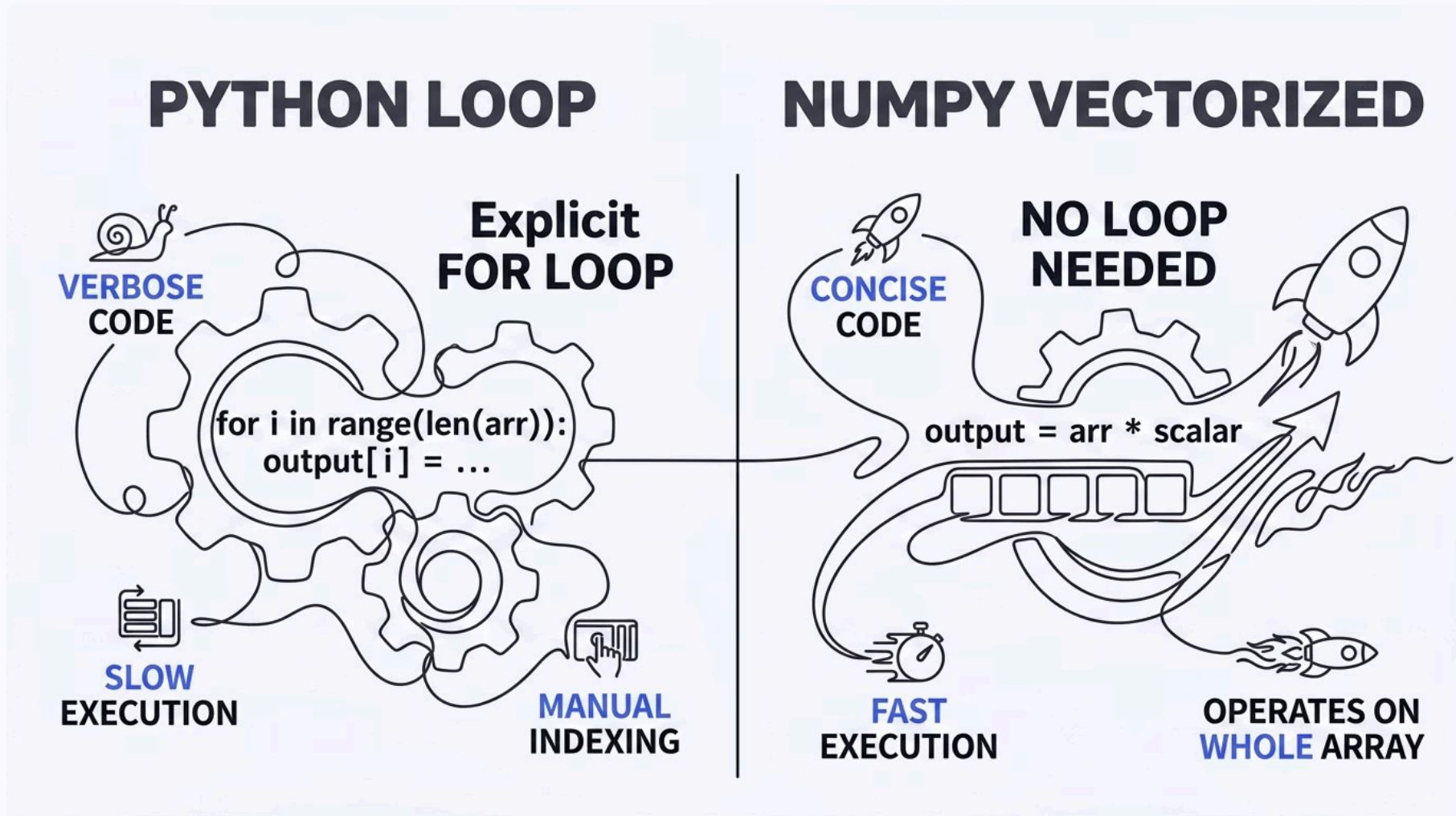
.max()/.min()

Maximum value: **42** · Minimum value: **4**

Output

```
ผลรวม: 108
เฉลี่ย: 18.0
ส่วนเบี่ยงเบนมาตรฐาน: 12.3153
สูงสุด/ต่ำสุด: 42 4
```

Vectorization vs. Python Loops



Vectorization is not just a convenience – for large arrays with millions of elements, NumPy's vectorized operations can be **10x to 100x faster** than equivalent Python loops, because the computation is delegated to optimized low-level C routines.

10.4 Two-Dimensional Arrays and Matrices

A two-dimensional NumPy array represents a **matrix**. It is created from a list of lists, where each inner list becomes one row. The `.shape` attribute returns a tuple (rows, columns). Individual elements are accessed using `[row, column]` indexing (zero-based).

Creating a 2D Array – Code Example

File: lOmatrix1.py

```
import numpy as np

M = np.array([[1, 2, 3],
              [4, 5, 6]])

print(M)
print("ขนาด:", M.shape)
print("สมาชิกแถว 0 หลัก 2:", M[0, 2])
```

Output

```
[[1 2 3]
 [4 5 6]]
ขนาด: (2, 3)
สมาชิกแถว 0 หลัก 2: 3
```

Understanding the 2D Array Structure

The matrix **M** has shape (2, 3) – meaning **2 rows** and **3 columns**.

Index	Col 0	Col 1	Col 2
Row 0	1	2	3
Row 1	4	5	6

Accessing **M[0, 2]** retrieves row 0, column 2 → value **3**.

10.5 Matrix Operations

NumPy supports full linear algebra operations on 2D arrays. The key distinction is between **element-wise** operations (using `*`) and true **matrix multiplication** (using the `@` operator).

Matrix Addition: $A + B$

Adds corresponding elements. Both matrices must have the same shape. Result is a matrix of the same shape.

Matrix Multiplication: $A @ B$

Performs true matrix multiplication (dot product). The number of columns in A must equal the number of rows in B. This is the standard mathematical matrix product.

Transpose: $A.T$

Flips the matrix over its diagonal – rows become columns and columns become rows. A matrix of shape (m, n) becomes shape (n, m).

Element-wise: $A * B$

Multiplies corresponding elements (Hadamard product). Different from matrix multiplication – both arrays must have the same shape.

Matrix Operations – Code Example

File: lOmatrix2.py

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("ผลบวก:\n", A + B)
print("ผลคูณเมทริกซ์:\n", A @ B)
print("ทรานสโพสของ A:\n", A.T)
```

Output

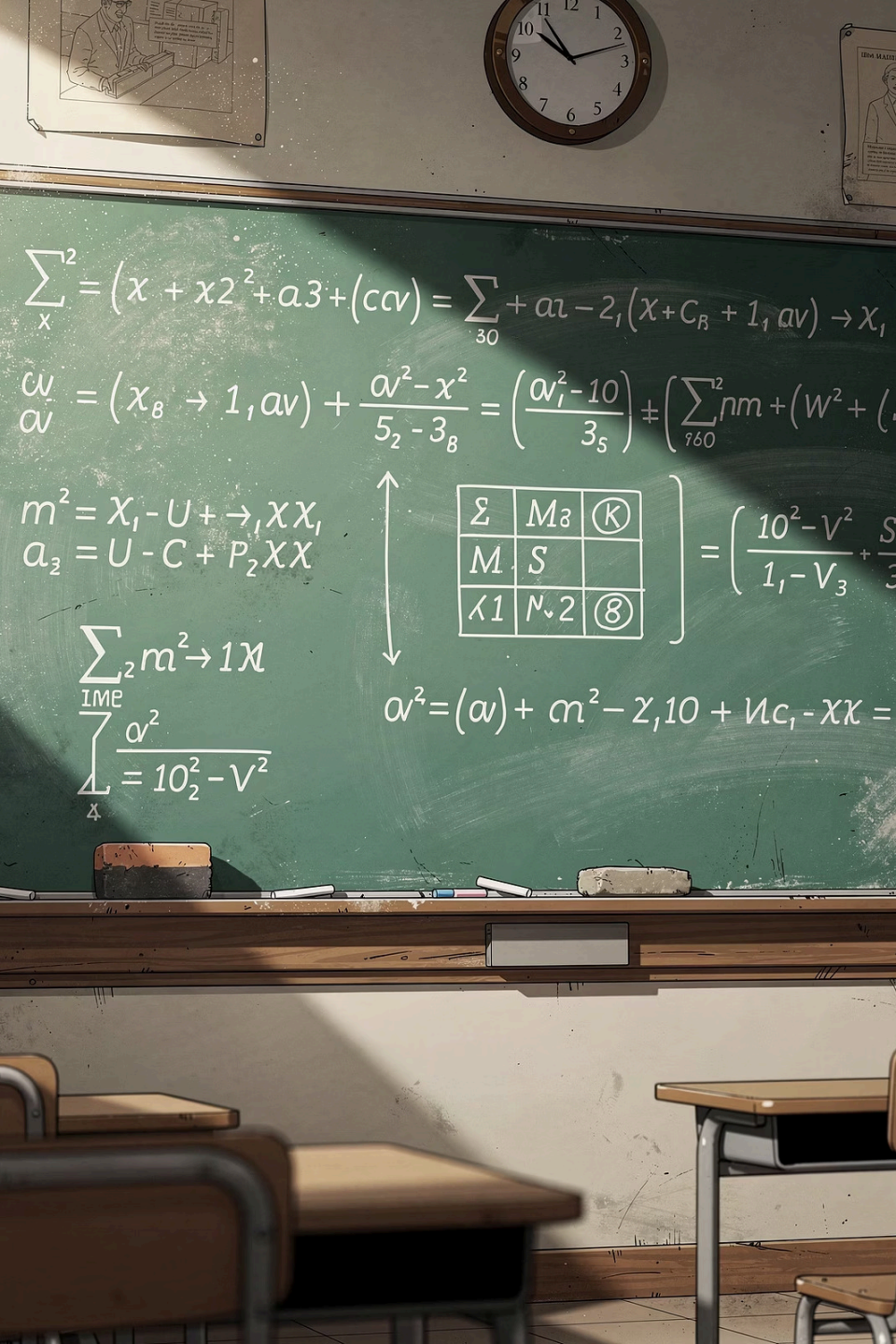
```
ผลบวก:
[[ 6  8]
 [10 12]]
ผลคูณเมทริกซ์:
[[19 22]
 [43 50]]
ทรานสโพสของ A:
[[1 3]
 [2 4]]
```

Verifying Matrix Multiplication Manually

For $A @ B$ where $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$:

- Row 0, Col 0: $(1 \times 5) + (2 \times 7) = 5 + 14 = \mathbf{19}$
- Row 0, Col 1: $(1 \times 6) + (2 \times 8) = 6 + 16 = \mathbf{22}$
- Row 1, Col 0: $(3 \times 5) + (4 \times 7) = 15 + 28 = \mathbf{43}$
- Row 1, Col 1: $(3 \times 6) + (4 \times 8) = 18 + 32 = \mathbf{50}$

 Use `@` for matrix multiplication. Using `*` instead gives element-wise multiplication – a common mistake!



10.6 Application: Solving Linear Systems

One of NumPy's most powerful applications is solving systems of linear equations of the form **Ax = b**, where A is the coefficient matrix, x is the vector of unknowns, and b is the right-hand side vector. The function `np.linalg.solve(A, b)` computes the exact solution efficiently.

Solving a 2×2 Linear System

Consider the system: $2x + y = 11$ and $x + 3y = 18$. We express this as matrix equation $Ax = b$ and solve with NumPy.

File: lOsolve.py

```
import numpy as np

A = np.array([[2, 1],
              [1, 3]])
b = np.array([11, 18])

x = np.linalg.solve(A, b)
print(f"x = {x[0]:.1f}, y = {x[1]:.1f}")
```

Output

```
x = 3.0, y = 5.0
```

Verification

Substituting $x = 3$, $y = 5$ back into the original equations:

- $2(3) + 5 = 6 + 5 = \mathbf{11} \checkmark$
- $3 + 3(5) = 3 + 15 = \mathbf{18} \checkmark$

👉 The solution $x = 3.0$, $y = 5.0$ satisfies both equations exactly.

Determinant and Matrix Inverse

NumPy's `linalg` module also provides functions to compute the **determinant** and **inverse** of a square matrix – fundamental tools in linear algebra.

File: `linalg.py`

```
import numpy as np

A = np.array([[2, 1],
              [1, 3]])

print("ดีเทอร์มิแนนต์:",
      np.linalg.det(A))
print("เมทริกซ์ผกผัน:\n",
      np.linalg.inv(A))
```

Output

```
ดีเทอร์มิแนนต์: 5.000000000000001
เมทริกซ์ผกผัน:
[[ 0.6 -0.2]
 [-0.2  0.4]]
```

Mathematical Interpretation

Determinant = 5

For a 2×2 matrix $\begin{bmatrix} a & b \\ c & d \end{bmatrix}$, $\det = ad - bc = (2 \times 3) - (1 \times 1) = 6 - 1 = \mathbf{5}$. A non-zero determinant confirms the system has a unique solution.

Inverse Matrix

The inverse A^{-1} satisfies $A \cdot A^{-1} = I$ (identity matrix). It can be used to solve $Ax = b$ as $x = A^{-1}b$. The small floating-point error in the determinant (5.000...001) is normal in numerical computing.

NumPy linalg Module Overview



`np.linalg.solve(A, b)`

Solves the linear system $Ax = b$ for x .
Requires A to be a square, non-singular matrix. Returns the solution vector x .



`np.linalg.det(A)`

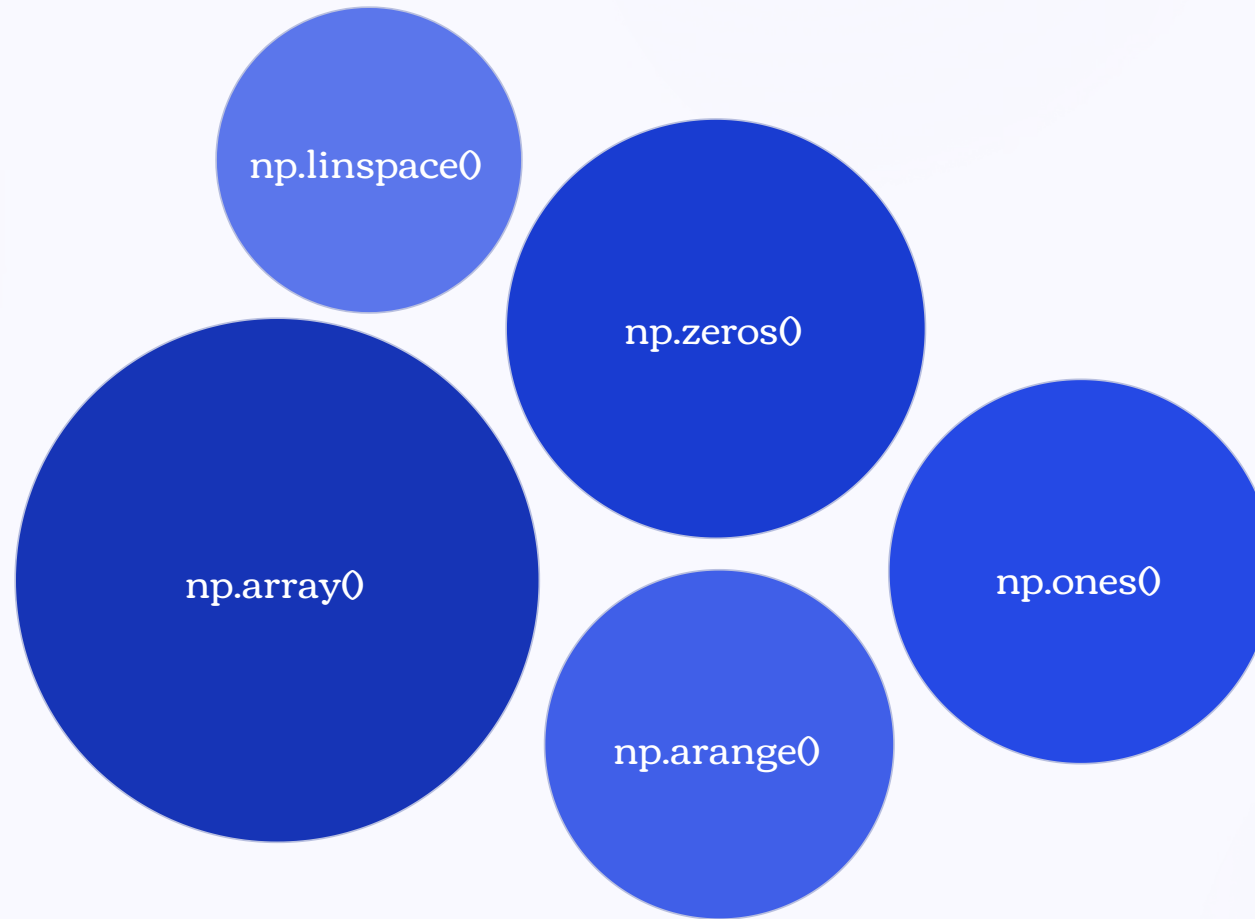
Computes the determinant of a square matrix. A determinant of zero means the matrix is singular and the system has no unique solution.



`np.linalg.inv(A)`

Computes the inverse of a square matrix.
Only exists when $\det(A) \neq 0$. Multiplying A by its inverse yields the identity matrix.

Summary of NumPy Array Creation Functions



Each creation function serves a distinct purpose: `array()` converts existing data, `zeros()` and `ones()` initialize arrays with constant values, `arange()` generates integer sequences like Python's `range()`, and `linspace()` produces precisely spaced floating-point values ideal for mathematical analysis and plotting.

10.7 Chapter Summary

Array Creation

Create arrays with `array`, `zeros`, `arange`, and `linspace` for different initialization needs.

Vectorized Computation

Perform element-wise math on entire arrays without loops. Use statistical methods like `.sum()`, `.mean()`, `.std()`.

Matrix Operations

Handle 2D arrays as matrices: addition (+), matrix multiplication (@), and transpose (.T).

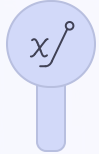
Linear Algebra

Solve linear systems with `np.linalg.solve`, compute determinants and inverses with `linalg.det` and `linalg.inv`.



Course Integration: The Full Journey

This chapter represents the culmination of the entire course – all prior knowledge converges into systematic numerical problem-solving:



Variables & Data Types

Storing and manipulating numerical values – the foundation of all computation.



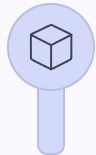
Conditions & Loops

Controlling program flow with `if` statements and `for/while` loops for repeated operations.



Data Structures

Lists, tuples, and dictionaries for organizing collections of data efficiently.



Functions

Encapsulating reusable logic into modular, callable units of code.



NumPy & Numerical Computing

Applying all prior skills to solve real mathematical problems with arrays, matrices, and linear algebra.

Exercises – Chapter 10

END-OF-CHAPTER PRACTICE

1

Exercise 1: Array Statistics

Create an array of integers 1 to 10. Display the sum, mean, and an array of the square of each element.

2

Exercise 2: Array Arithmetic

Create two arrays and display their element-wise sum, difference, and product.

3

Exercise 3: Matrix Transpose

Create a 3×3 matrix, then display its transpose and the sum of all elements.

4

Exercise 4: Solve Linear System

Use NumPy to solve the system: $3\mathbf{x} + 2\mathbf{y} = 12$ and $\mathbf{x} - \mathbf{y} = 1$.

5

Exercise 5: Sine Wave

Create an array of 100 equally spaced values in $[0, 2\pi]$, then compute the sine of each value.

Exercise 1 Solution – Array Statistics

Code

```
import numpy as np

a = np.arange(1, 11)
print("ผลรวม:", a.sum())
print("เฉลี่ย:", a.mean())
print("กำลังสอง:", a ** 2)
```

Output

```
ผลรวม: 55
เฉลี่ย: 5.5
กำลังสอง: [ 1  4  9 16 25
          36 49 64 81 100]
```

Explanation

- `np.arange(1, 11)` creates the array `[1, 2, 3, ..., 10]`.
- `a.sum()` computes $1+2+\dots+10 = \mathbf{55}$ (arithmetic series formula: $n(n+1)/2 = 10 \times 11/2$).
- `a.mean()` = $55 \div 10 = \mathbf{5.5}$.
- `a ** 2` squares each element vectorially: `[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]`.

✓ No loops needed – all three results computed with single-line vectorized expressions.

Exercise 4 Solution – Solving a Linear System

Solve the system: $3x + 2y = 12$ and $x - y = 1$

Code

```
import numpy as np

A = np.array([[3, 2],
              [1, -1]])
b = np.array([12, 1])

x = np.linalg.solve(A, b)
print(f"x = {x[0]:.1f}, y = {x[1]:.1f}")
```

Output

```
x = 2.8, y = 1.8
```

Verification

Substituting $x = 2.8$, $y = 1.8$:

- $3(2.8) + 2(1.8) = 8.4 + 3.6 = 12 \checkmark$
- $2.8 - 1.8 = 1 \checkmark$

The coefficient matrix $A = \begin{bmatrix} 3 & 2 \\ 1 & -1 \end{bmatrix}$ encodes the left-hand side coefficients. The vector $b = \begin{bmatrix} 12 \\ 1 \end{bmatrix}$ encodes the right-hand side constants. `np.linalg.solve` finds the unique solution when $\det(A) \neq 0$.

Exercise 5 Solution – Sine Wave Computation

Create 100 equally spaced values in $[0, 2\pi]$ and compute their sine values.

Code

```
import numpy as np

x = np.linspace(0, 2 * np.pi, 100)
y = np.sin(x)

print("ค่าแรก 3 ตัว:", y[:3])
print("ค่ามากที่สุด:", round(y.max(), 4))
```

Output

```
ค่าแรก 3 ตัว: [0.      0.06342392
              0.12659245]
ค่ามากที่สุด: 0.9999
```

Key Points

- `np.pi` is NumPy's built-in constant $\pi \approx 3.14159...$
- `np.linspace(0, 2*np.pi, 100)` generates 100 points from 0 to 2π inclusive.
- `np.sin(x)` applies the sine function to all 100 values simultaneously – a vectorized universal function (ufunc).
- The maximum value is 0.9999 (not exactly 1.0) because none of the 100 sample points falls exactly at $\pi/2$ where $\sin = 1$.

 This pattern – `linspace` + `ufunc` – is the standard NumPy approach for plotting mathematical functions.

NumPy Array Attributes Quick Reference

Attribute / Method	Description	Example
<code>.shape</code>	Tuple of array dimensions	<code>M.shape</code> → (2, 3)
<code>.size</code>	Total number of elements	<code>a.size</code> → 5
<code>.dtype</code>	Data type of elements	<code>a.dtype</code> → int64
<code>.sum()</code>	Sum of all elements	<code>a.sum()</code> → 55
<code>.mean()</code>	Arithmetic mean	<code>a.mean()</code> → 5.5
<code>.std()</code>	Standard deviation	<code>data.std()</code> → 12.3153
<code>.max()</code> / <code>.min()</code>	Maximum / minimum value	<code>data.max()</code> → 42
<code>.T</code>	Transpose of the array	<code>A.T</code> → flipped rows/cols

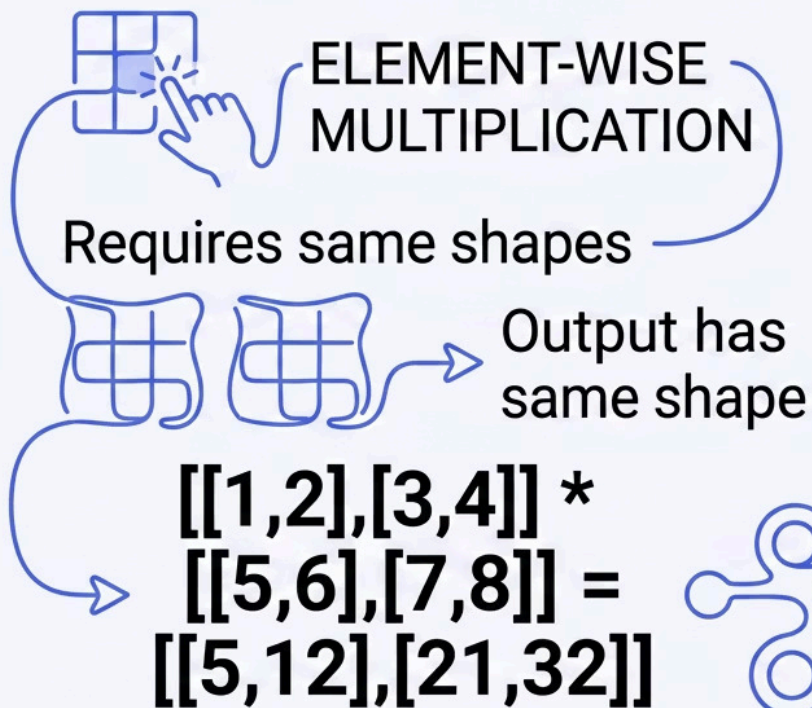
NumPy linalg Functions Quick Reference

Function	Description	Requirement
<code>np.linalg.solve(A, b)</code>	Solve linear system $Ax = b$	A must be square and non-singular
<code>np.linalg.det(A)</code>	Compute determinant of A	A must be square
<code>np.linalg.inv(A)</code>	Compute inverse of A	$\det(A) \neq 0$

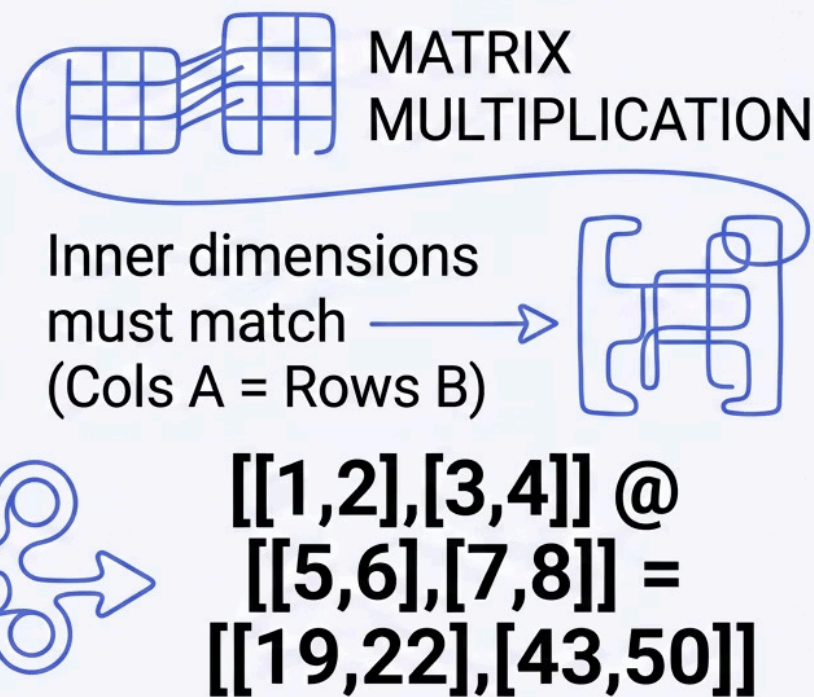
⚠ If $\det(A) = 0$, the matrix is **singular** – it has no inverse and the linear system either has no solution or infinitely many solutions. `np.linalg.solve` will raise a `LinAlgError` in this case.

Operator Reference: * vs @ in NumPy

* Operator



@ Operator



This distinction is one of the most important concepts in NumPy matrix programming. Always use @ when you intend standard linear algebra matrix multiplication, and * only when you want element-by-element products.

Understanding np.linspace for Mathematical Functions

Syntax

```
np.linspace(start, stop, num)
```

Example: 5 points in $[0, 1]$

```
np.linspace(0, 1, 5)  
# → [0.  0.25 0.5  0.75 1.]
```

Example: 100 points in $[0, 2\pi]$

```
x = np.linspace(0, 2*np.pi, 100)  
y = np.sin(x)
```

When to Use linspace vs arange

Use linspace when you need a fixed count

You specify exactly how many points you want. Ideal for plotting, numerical integration, and function evaluation where uniform coverage matters more than step size.

Use arange when you need a fixed step

You specify the step increment. Ideal for integer sequences or when the interval between values is the meaningful quantity, similar to Python's `range()`.

NumPy Universal Functions (ufuncs)

NumPy provides a large library of **universal functions (ufuncs)** – mathematical functions that operate element-wise on arrays, enabling vectorized computation without loops.

Math Functions

`np.sqrt(a)` – square root

`np.abs(a)` – absolute value

`np.exp(a)` – exponential e^x

`np.log(a)` – natural logarithm

Trigonometric

`np.sin(x)` – sine

`np.cos(x)` – cosine

`np.tan(x)` – tangent

`np.pi` – constant π

Rounding

`np.round(a, n)` – round to n decimals

`np.floor(a)` – floor function

`np.ceil(a)` – ceiling function

Aggregation

`np.sum(a)` – total sum

`np.cumsum(a)` – cumulative sum

`np.prod(a)` – product of all elements

Broadcasting in NumPy

Broadcasting is NumPy's mechanism for performing operations between arrays of different shapes. When you write `a * 2`, the scalar 2 is "broadcast" to match the shape of array `a`.

Scalar Broadcasting

```
a = np.array([1, 2, 3, 4])  
a * 2 # → [2, 4, 6, 8]  
a + 10 # → [11, 12, 13, 14]  
a ** 2 # → [1, 4, 9, 16]
```

The scalar is applied to every element – equivalent to multiplying by an array of the same value.

Why Broadcasting Matters

- Eliminates the need to manually resize arrays before operations.
- Makes code more concise and readable.
- Enables efficient computation without creating large intermediate arrays in memory.
- Applies to all arithmetic operators: `+`, `-`, `*`, `/`, `**`.

Indexing and Slicing 2D Arrays

Two-dimensional NumPy arrays support powerful indexing and slicing to access rows, columns, or sub-matrices.

```
import numpy as np

M = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

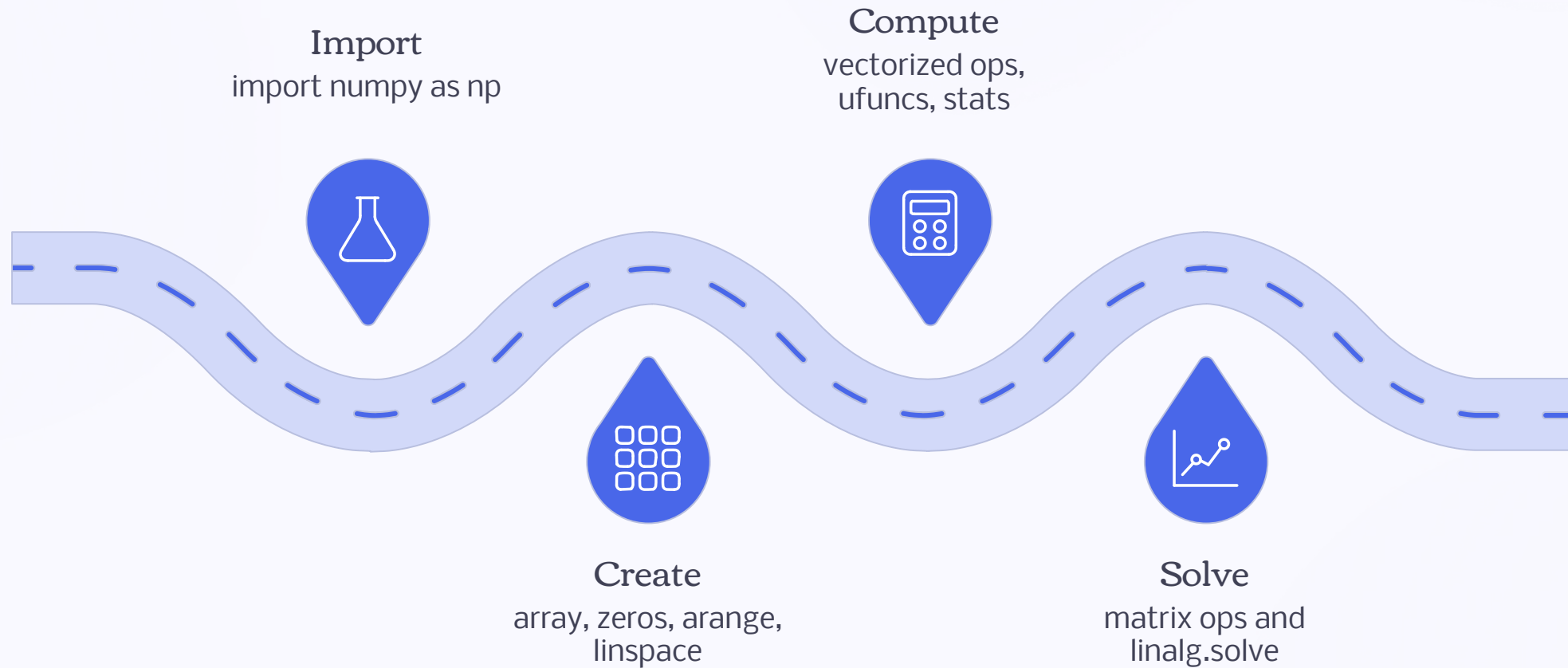
print(M[1, 2])  # → 6
print(M[0, :])  # → [1 2 3] (row 0)
print(M[:, 1])  # → [2 5 8] (col 1)
print(M[0:2, 1:]) # → [[2 3]
                      #   [5 6]]
```

Indexing Syntax Summary

Expression	Meaning
<code>M[i, j]</code>	Single element at row <i>i</i> , col <i>j</i>
<code>M[i, :]</code>	Entire row <i>i</i>
<code>M[:, j]</code>	Entire column <i>j</i>
<code>M[r1:r2, c1:c2]</code>	Sub-matrix (slice)

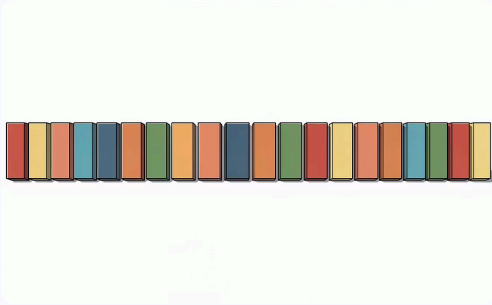
 NumPy uses zero-based indexing. The colon `:` means "all" along that axis.

Complete NumPy Workflow Summary



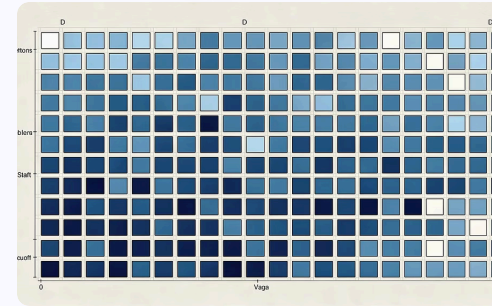
Every NumPy program follows this four-stage pattern: import the library, create the necessary arrays, apply vectorized computations or mathematical functions, and use the results to solve real problems. This workflow scales from simple statistics on a 5-element array to solving large systems of linear equations with hundreds of variables.

Key Concepts: NumPy vs Python Lists



Python List

- Can hold mixed data types
- Operations require explicit loops
- Slower for large numerical datasets
- No built-in math functions
- General-purpose container



NumPy ndarray

- Homogeneous data type only
- Vectorized operations – no loops needed
- Significantly faster for numerical work
- Rich built-in math and statistics
- Designed for scientific computing



NumPy in the Broader Python Ecosystem

NumPy is the **foundation** of the entire Python scientific computing stack. Understanding NumPy arrays is a prerequisite for working with higher-level libraries used in data science, machine learning, and engineering:

Pandas

Data analysis library built on NumPy arrays. DataFrames are essentially labeled 2D NumPy arrays with powerful data manipulation tools.

Matplotlib

Plotting library that accepts NumPy arrays directly. The `linspace + sin` pattern from Exercise 5 is the standard way to plot mathematical functions.

SciPy

Advanced scientific computing extending NumPy with optimization, integration, signal processing, and more linear algebra tools.

Scikit-learn

Machine learning library where all data is represented as NumPy arrays. Every ML algorithm operates on ndarray inputs and outputs.

Common Mistakes to Avoid

Using * Instead of @ for Matrix Multiplication

`A * B` performs element-wise multiplication (Hadamard product), not matrix multiplication. Always use `A @ B` for the standard linear algebra matrix product. This is one of the most frequent errors for beginners.

Confusing arange and linspace

`np.arange(0, 1, 0.1)` may not produce exactly 10 values due to floating-point precision. Use `np.linspace(0, 1, 10)` when you need a guaranteed count of evenly spaced points.

Solving a Singular System

Calling `np.linalg.solve(A, b)` when `det(A) = 0` raises a `LinAlgError`. Always check the determinant first if the system's solvability is uncertain.

Modifying Arrays In-Place Unexpectedly

NumPy arrays are mutable. Operations like `a += 1` modify the original array. Use `b = a + 1` to create a new array and preserve the original.

Chapter 10 at a Glance

5

Creation Functions

array, zeros, ones, arange, linspace

4

Statistical Methods

sum, mean, std, max/min

3

Matrix Operations

Addition (+), multiplication (@), transpose
(.T)

3

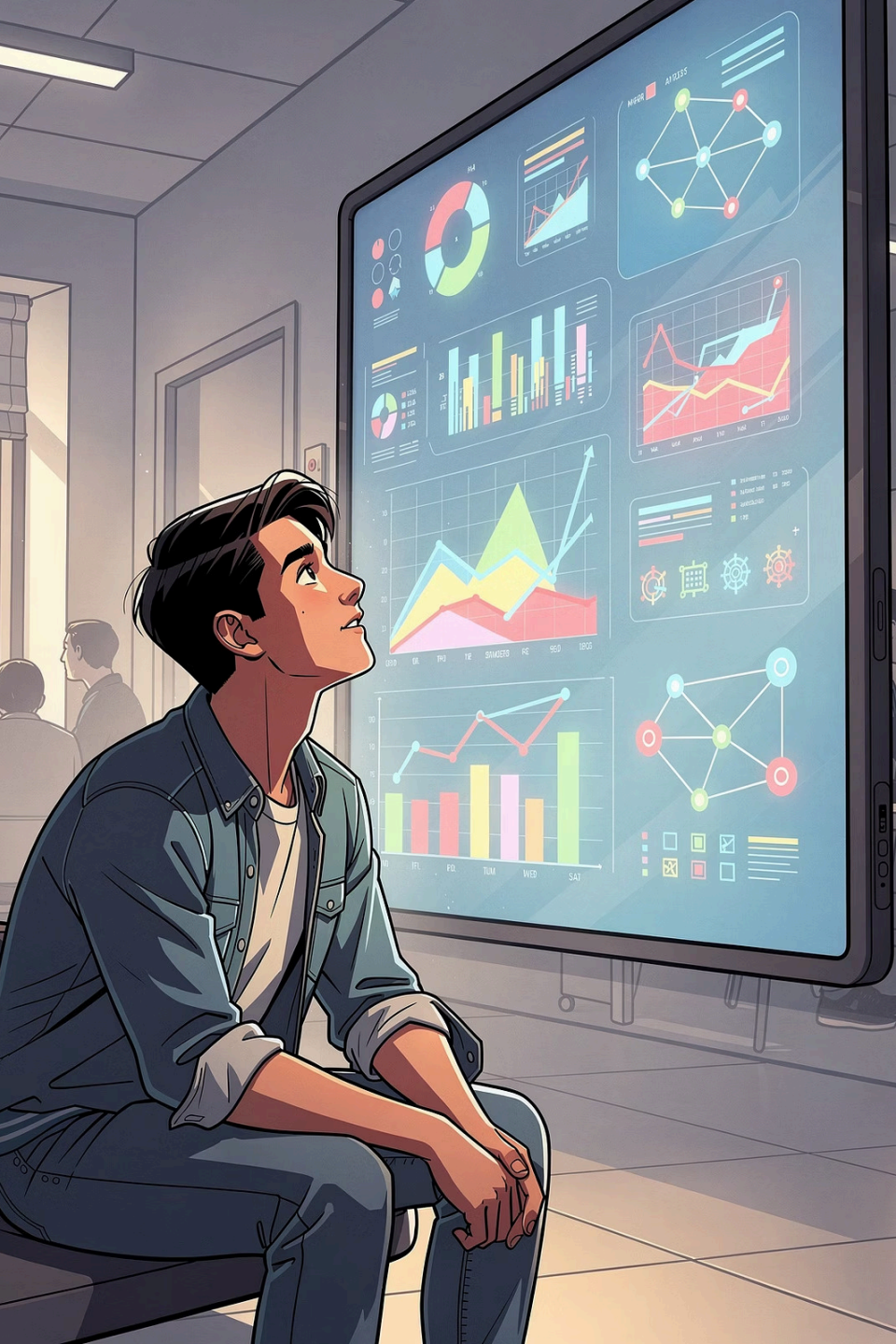
linalg Functions

solve, det, inv

5

Exercises

Covering arrays, statistics, matrices, and
linear systems



Looking Ahead: Beyond This Course

This chapter – and this course – establishes the **essential foundation** for advanced study in computational mathematics and data science. The skills acquired here directly enable:

- **Scientific Computing**
Numerical methods for differential equations, optimization, and simulation using SciPy and NumPy together.
- **Data Science & Analysis**
Manipulating and analyzing large datasets with Pandas, built directly on NumPy's array infrastructure.
- **Machine Learning**
Every machine learning algorithm – from linear regression to neural networks – operates on NumPy arrays as its fundamental data representation.
- **Mathematical Visualization**
Plotting functions, surfaces, and data with Matplotlib using the linspace-and-ufunc patterns learned in this chapter.



Course Complete – Computer Programming for Mathematics 1

Congratulations on completing **040223101 Computer Programming for Mathematics**

1. Over 10 chapters, you have built a complete foundation in Python programming applied to mathematical problem-solving.

Core Python

Variables, data types, conditions, loops, functions, and data structures – the building blocks of all programs.

Mathematical Application

Applying programming constructs to solve real mathematical problems systematically and efficiently.

NumPy Mastery

Arrays, vectorization, matrix operations, and linear algebra – the gateway to scientific computing and data science.



This course aligns with **CLO 5**. The knowledge and skills gained here form a critical foundation for all higher-level courses in computational science, data analysis, and applied mathematics.